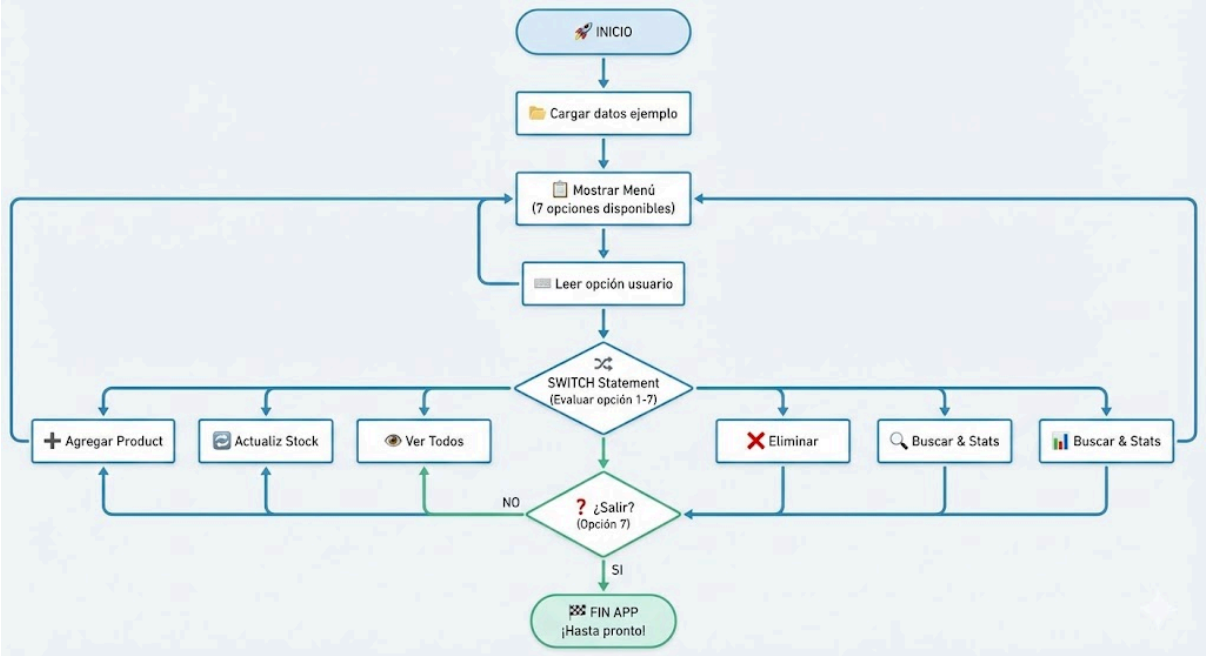
ESQUEMA DE DISEÑO

FLUJO PRINCIPAL



ESTRUCTURAS DE CÓDIGO UTILIZADAS

Estructuras de Control

IF-ELSE:

- Validación de entradas del usuario
- Verificación de stock suficiente
- Confirmación de operaciones
- Alertas de stock bajo en DisplayInfo()

SWITCH:

- Selección de opciones del menú principal
- Selección de tipo de actualización de stock
- Selección de tipo de búsqueda

Bucles

FOR:

- Iterar sobre todos los productos para mostrarlos
- Mostrar resultados de búsqueda
- Listar productos con stock bajo

WHILE:

- Bucle principal de la aplicación (menú)
- Cálculo de estadísticas

FOREACH:

- Buscar productos por ID
- Buscar productos por nombre
- Búsquedas en general

Métodos Personalizados

Clase Product:

- DisplayInfo() → Mostrar información del producto
- GetTotalValue() → Calcular valor total

Clase Program:

- Main() → Punto de entrada
- DisplayMenu() → Mostrar menú
- AddNewProduct() → Agregar producto
- UpdateStock() → Actualizar stock
- AddStock() → Agregar unidades
- ReduceStock() → Reducir unidades
- SetStock() → Establecer stock
- ViewAllProducts() → Ver todos
- DeleteProduct() → Eliminar producto
- SearchProduct() → Buscar (menú)
- SearchById() → Buscar por ID
- SearchByName() → Buscar por nombre
- ShowStatistics() → Mostrar estadísticas
- FindProductById() → Buscar producto por ID
- AddSampleProducts() → Datos de ejemplo

Estructuras de datos

Clase Product:

- Id: int
- Name: string
- Price: decimal
- Stock: int

Variables globales:

- inventory: List<Product> → Almacena todos los productos
- nextId: int → Contador para IDs únicos

VALIDACIONES IMPLEMENTADAS

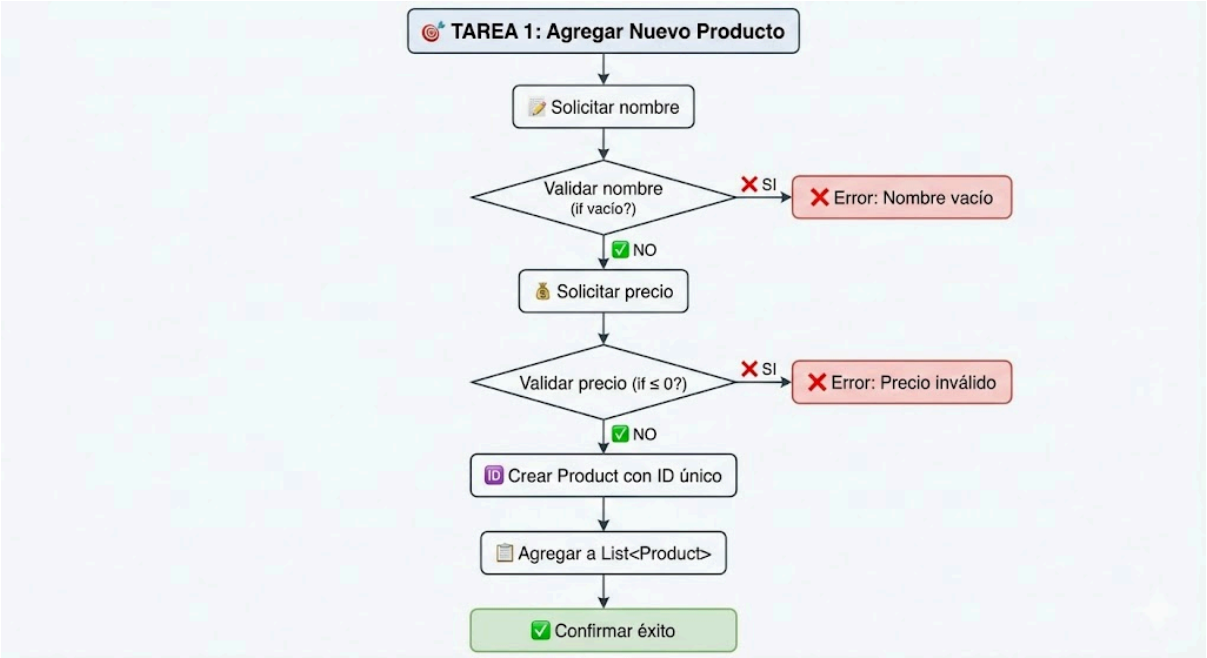
1. Nombre no vacío
2. Precio positivo
3. Stock no negativo
4. ID válido para operaciones
5. Stock suficiente para reducción
6. Producto existe antes de operar
7. Confirmación para eliminar
8. Término de búsqueda no vacío

DETALLE

TAREA 1: Agregar Nuevo Producto

Componentes de código:

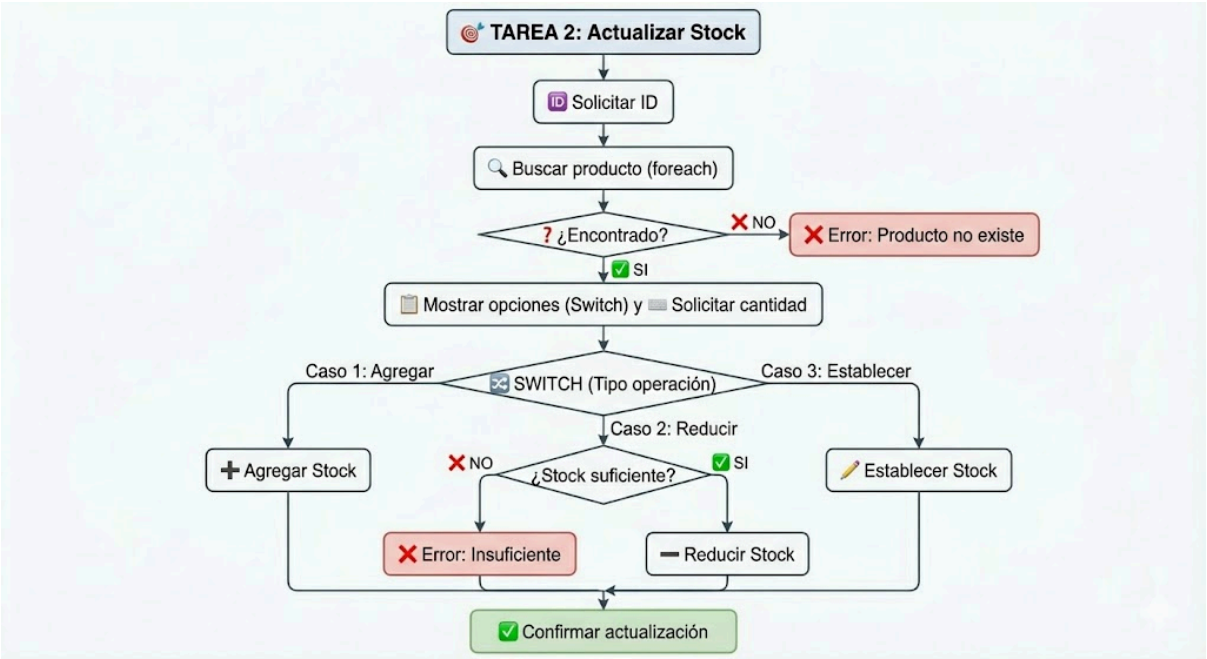
- **Variables**: `string name`, `decimal price`, `int stock`, `int nextId`
- **Estructuras de control**:
  - `if-else` para validar nombre vacío
  - `if` para validar precio positivo
  - `if` para validar stock no negativo
- **Métodos**: `AddNewProduct()`
- **Objetos**: Crear nueva instancia de `Product`



TAREA 2: Actualizar Stock

Componentes de código:

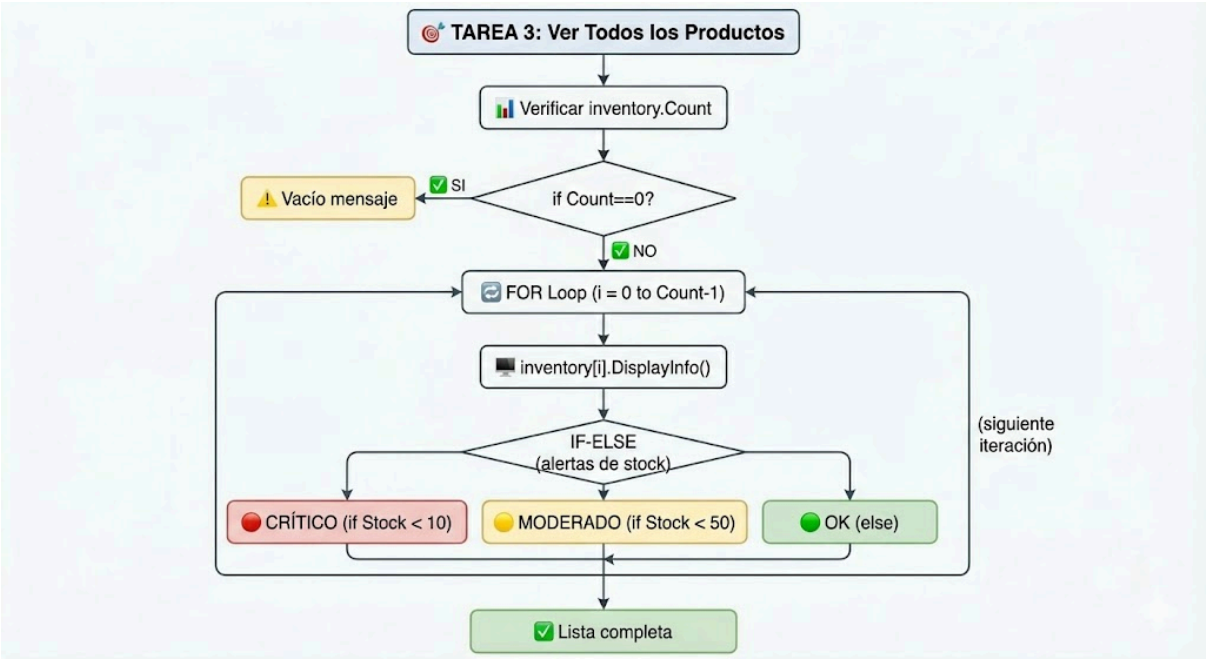
- **Variables**: `int id`, `int quantity`, `string option`
- **Estructuras de control**:
  - `switch` para tipo de actualización (agregar/reducir/establecer)
  - `if-else` para validar stock suficiente
- **Bucles**: `foreach` para buscar producto por ID
- **Métodos**: `UpdateStock()`, `AddStock()`, `ReduceStock()`, `SetStock()`



TAREA 3: Ver Todos los Productos

Componentes de código:

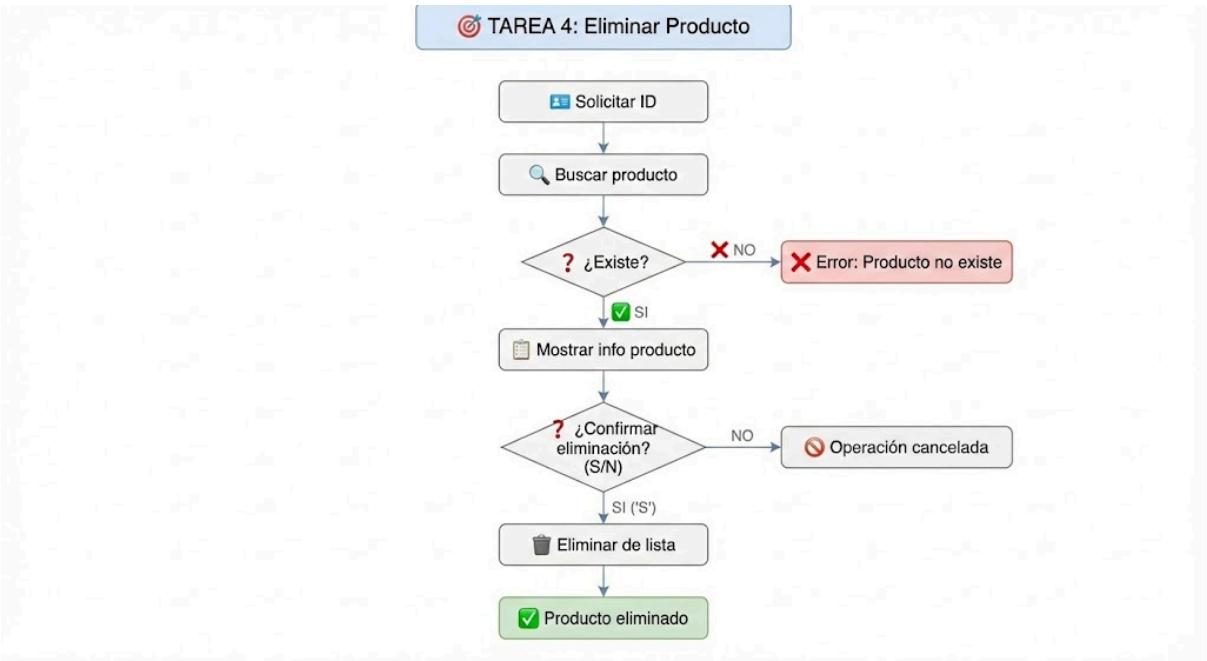
- **Variables**: `int i` (contador)
- **Estructuras de control**:
  - `if` para verificar lista vacía
  - `if-else` en método `DisplayInfo()` para alertas de stock
- **Bucles**: `for` para iterar sobre todos los productos
- **Métodos**: `ViewAllProducts()`, `Product.DisplayInfo()`



TAREA 4: Eliminar Producto

Componentes de código:

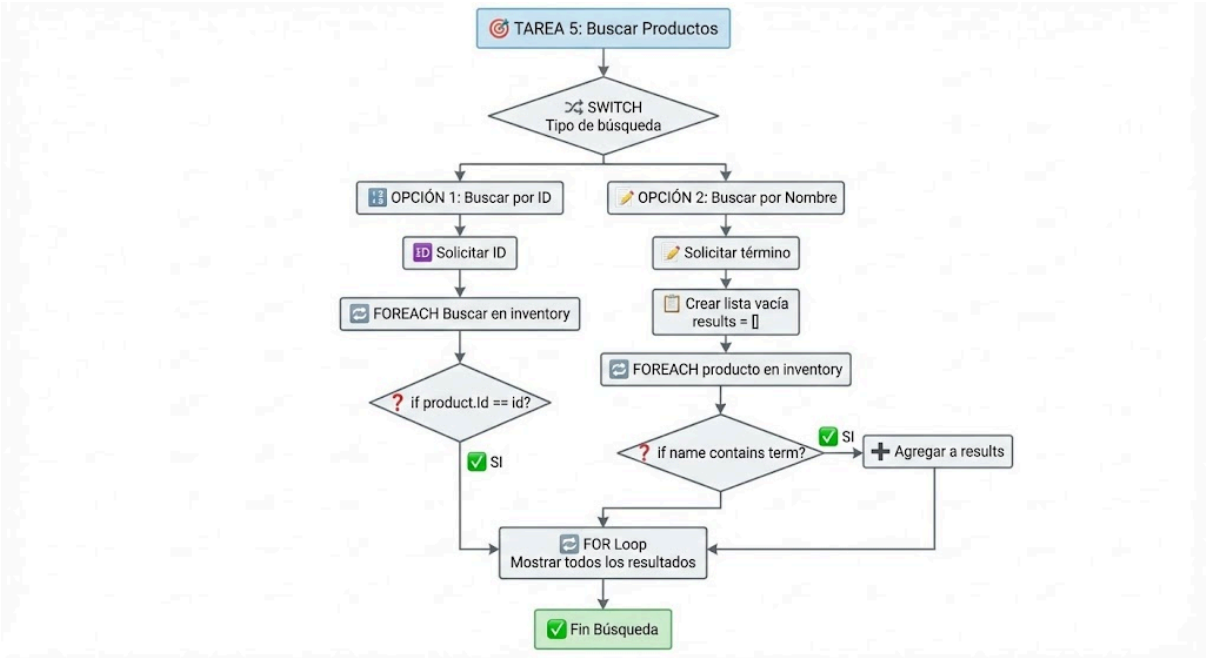
- **Variables**: `int id`, `string confirmation`
- **Estructuras de control**:
  - `if` para validar existencia
  - `if-else` para confirmar eliminación
- **Bucles**: `foreach` implícito en búsqueda
- **Métodos**: `DeleteProduct()`, `FindProductById()`



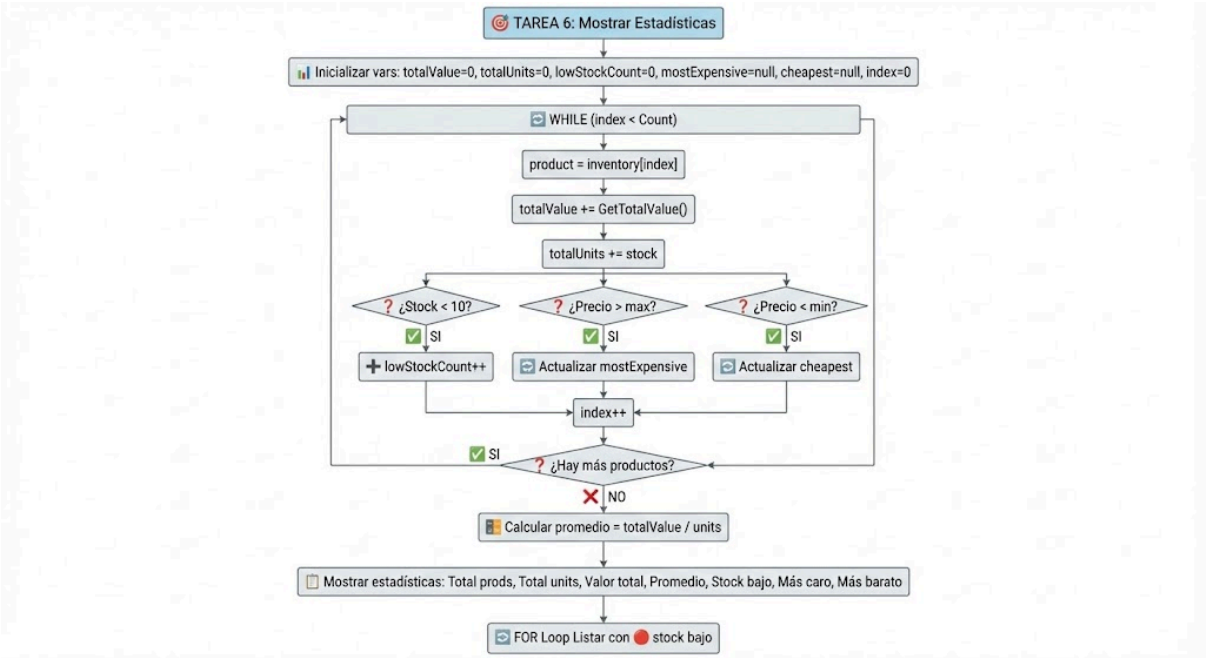
TAREA 5: Buscar Productos

Componentes de código:

- **Variables**: `int id`, `string searchTerm`, `List<Product> results`
- **Estructuras de control**:
  - `switch` para tipo de búsqueda (por ID o nombre)
  - `if` para verificar resultados encontrados
- **Bucles**:
  - `foreach` para buscar por nombre
  - `for` para mostrar resultados
- **Métodos**: `SearchProduct()`, `SearchById()`, `SearchByName()`



TAREA 6: Mostrar Estadísticas

Componentes de código:

- **Variables**: `decimal totalValue`, `int totalUnits`, `int lowStockCount`, `int index`
- **Estructuras de control**:
  - `if` para actualizar producto más caro/barato
  - `if` para contar stock bajo
- **Bucles**:
  - `while` para calcular estadísticas
  - `for` para mostrar productos con stock bajo
- **Métodos**: `ShowStatistics()`, `Product.GetTotalValue()`